

EduMerge Student Support and Ticket Management System

Complete Project Documentation

EduMerge is a full-stack campus support portal. Students use it to submit and track support requests, support staff use it to triage and resolve tickets, and administrators use it to monitor service performance, workloads, and SLA compliance.

The system combines role-based dashboards, ticket workflows, automatic SLA calculation, duplicate-ticket detection, comments, internal staff notes, assignment management, analytics, and a complete ticket activity history.

1. What Is This Project?

This project is a student support and help-desk application designed for an educational institution. It provides one place to manage issues related to academics, hostel services, transport, fees, IT, and general requests.

The application supports three roles:

- Student: Creates tickets, views ticket progress, receives updates, and reopens resolved tickets when necessary.
- Staff: Reviews assigned and unassigned queues, changes ticket status, responds to students, adds internal notes, updates priorities, and manages SLA-breached tickets.
- Admin: Views all tickets, assigns work, monitors analytics, and reviews staff workload and SLA performance.

2. Main Features

Student Features

- Create a support ticket with a title, description, category, and priority.
- Receive an automatically calculated SLA target.
- Detect similar open tickets while entering a subject.
- Search and filter personal tickets.
- View ticket status, assigned staff, SLA information, comments, and activity history.
- Reopen a resolved ticket when the issue is not fully solved.

Staff Features

- View personal, unassigned, and SLA-breached ticket queues.
- Assign or reassign tickets.
- Move tickets through OPEN, ASSIGNED, IN_PROGRESS, PENDING_STUDENT_INFO, RESOLVED, and CLOSED states.
- Add public responses and private internal notes.
- Change ticket priority and recalculate its SLA deadline.
- Track ticket history and audit events.

Admin Features

- View the complete system ticket list.
- Monitor total volume, active tickets, SLA compliance, and SLA breaches.
- View ticket volume by category.
- Review staff workload, active assignments, resolutions, and breaches.
- Manage assignment and operational support performance.

3. SLA and Audit Features

Tickets receive an SLA target based on priority:

Priority	SLA Target
Critical	4 hours
High	8 hours
Medium	24 hours
Low	48 hours

The backend checks active tickets every 60 seconds. When a ticket passes its due date, it is marked as breached and an internal activity record is created. Ticket activities also record creation, assignment, status changes, priority changes, comments, reopening, and other important events.

4. Technology Stack

Frontend

- React 18 for the user interface and component-based application structure.
- Vite for local development and production bundling.
- Tailwind CSS for responsive styling and utility classes.
- Axios for HTTP requests to the backend API.
- Lucide React for interface icons.
- Recharts for administrator analytics charts.

Backend

- Node.js as the runtime environment.
- Express for the REST API server and routing.
- Prisma as the database ORM and query layer.
- SQLite as the local relational database.
- JWT for authentication tokens.
- bcryptjs for password hashing.
- Multer for multipart and attachment-related request handling.
- CORS and dotenv for server configuration and request handling.

Deployment and tools include Docker, Docker Compose, Prisma CLI, and the Prisma seed script.

5. How the Frontend Works

The frontend is located in the client/ directory.

Application Entry Point

client/src/main.jsx starts the React application. client/src/App.jsx is the main application shell. It reads authentication state, displays navigation, selects the dashboard based on the logged-in user's role, manages the active tab, and opens the create-ticket and ticket-detail modals.

Authentication

AuthContext.jsx stores the current user and authentication state. After login, the JWT is stored in browser localStorage under edumerge_token. The shared Axios client in client/src/services/api.js reads this token before each request and sends it as a Bearer token.

Role-Based Dashboards

- StudentDashboard.jsx displays the student's tickets and ticket statistics.
- StaffDashboard.jsx displays work queues and ticket-management controls.
- AdminDashboard.jsx displays analytics, workload information, and all tickets.

Reusable components such as TicketList, StatusBadge, PriorityBadge, SlaBadge, CreateTicketModal, and TicketDetailModal keep common behavior consistent across dashboards.

Frontend Request Flow

- A user interacts with a dashboard or modal.
- The component calls the Axios API client.
- Axios sends a request to an /api endpoint and attaches the JWT.
- The backend returns JSON data or an error.
- React updates component state and re-renders the relevant view.

6. How the Backend Works

The backend is located in the server/ directory.

server/src/index.js creates the Express application and configures CORS, JSON parsing, the health check, authentication routes, ticket routes, analytics routes, user routes, a global error handler, and the periodic SLA checker.

Backend Request Flow

- The frontend sends an HTTP request to an API endpoint.
- Express matches the request to a route.
- Authentication middleware validates the JWT and identifies the user.
- Role checks restrict operations to the correct user types.
- A controller validates input and performs the requested operation.
- Prisma reads or writes data in SQLite.
- The controller returns a JSON response to the frontend.

Main Backend Modules

- authController.js: Login and current-user authentication logic.
- ticketController.js: Ticket creation, filtering, status changes, assignment, comments, priority updates, and reopening.

- analyticsController.js: Dashboard statistics and staff workload data.
- userController.js: User-related operations.
- middleware/auth.js: JWT authentication and role-based access control.
- services/slaService.js: SLA calculation and scheduled breach detection.
- config/prisma.js: Shared Prisma client configuration.
- prisma/seed.js: Demo users and ticket data.

7. Where Is the Data Stored?

The application stores relational data in a local SQLite database managed by Prisma. The database URL in `server/prisma/schema.prisma` is `file:./dev.db`. Because the schema is inside `server/prisma/`, the database file is normally created at `server/prisma/dev.db`.

Database Models

- User: Stores student, staff, and admin accounts, including email, hashed password, name, role, department, phone, avatar, and timestamps.
- Ticket: Stores ticket code, title, description, category, priority, status, SLA hours, due date, breach status, student ownership, staff assignment, and timestamps.
- TicketActivity: Stores the ticket audit timeline, actor, action, changed values, comments, internal-note status, and creation time.
- Attachment: Stores attachment metadata such as ticket, file name, file URL, MIME type, size, and creation time.

The database can be inspected with Prisma Studio using:

```
cd server
npm run db:studio
```

8. API Overview

Method	Endpoint	Purpose
GET	/api/health	Check server status
POST	/api/auth/login	Authenticate a user
GET	/api/auth/me	Return current user
POST	/api/tickets	Create a ticket
GET	/api/tickets	List scoped and filtered tickets
GET	/api/tickets/:id	View ticket and activity history
PATCH	/api/tickets/:id/status	Change ticket status
PATCH	/api/tickets/:id/assign	Assign or reassign staff
PATCH	/api/tickets/:id/priority	Change priority and SLA
POST	/api/tickets/:id/comments	Add comment or internal note
POST	/api/tickets/:id/reopen	Reopen a resolved ticket
GET	/api/analytics/dashboard	Load metrics and workload data

9. Local Setup

Prerequisites: Node.js 18 or newer, npm 9 or newer, and optionally Docker.

Start the backend:

```
cd server
npm install
npx prisma db push
npm run db:seed
npm start
```

The backend runs on `http://localhost:5000` by default. For development with automatic restarts, use `npm run dev`.

Start the frontend in another terminal:

```
cd client
npm install
npm run dev
```

Build the frontend with `npm run build`. To run the complete stack with Docker, execute `docker-compose up --build` from the repository root.

10. Demo Accounts

Role	Email	Password
Student	student@edumerge.com	password123
Student	student2@edumerge.com	password123
Staff	agent.hostel@edumerge.com	password123
Staff	agent.it@edumerge.com	password123
Staff	agent.academics@edumerge.com	password123
Admin	admin@edumerge.com	password123

11. Project Structure

```
Student/
|- client/
|  |- src/components/      Reusable UI components
|  |- src/context/         Authentication state
|  |- src/services/        Axios API client
|  |- src/App.jsx           Main application shell
|  `-- src/index.css        Global styles
|- server/
|  |- prisma/schema.prisma  Database models and SQLite configuration
|  |- prisma/dev.db         Local SQLite database after setup
|  |- src/controllers/      Request and business logic
|  |- src/middleware/       Authentication and authorization
|  |- src/routes/           API route definitions
|  |- src/services/         SLA and supporting services
|  `-- src/index.js         Express server entry point
|- Dockerfile
|- docker-compose.yml
|- README.md
`- PROJECT_DOCUMENTATION.pdf
```

12. Brief Project Summary

EduMerge is an end-to-end ticket management platform for campus support operations. The React frontend provides role-specific workflows and dashboards. The Express backend exposes secure REST APIs and applies business rules. Prisma connects the backend to the SQLite database, while the SLA service continuously monitors unresolved tickets and records breaches. Together, these parts provide a complete support lifecycle from ticket

submission through assignment, communication, resolution, auditing, and reporting.